

LibSignal Research

Team Instructions

Collective Intelligence Project

Salman · Rashid · Madina

June 1, 2026

Repository: <https://github.com/sal0-h/LibSignal>

Framework: LibSignal — Reinforcement Learning for Traffic Signal Control

Contents

1 Overview	2
1.1 Key Concepts	2
2 Google Colab Setup	2
2.1 Step 1: Open the Notebook	2
2.2 Step 2: Change Your Name	2
2.3 Step 3: Run the Setup Cell	2
2.4 Where Do Outputs Go?	3
3 Running Experiments	3
3.1 Option 1: Edit the Config File Directly (Recommended)	3
3.2 Option 2: Smart Config Injector Cell	4
4 Command-Line Arguments Reference	4
5 Available Agents	5
6 Available SUMO Networks	5
7 Configuration (YAML) Reference	5
7.1 Key Trainer Parameters	6
7.2 Key Model (Agent) Parameters	6
7.3 Logger Parameters	6
8 Team Member Assignments	6
9 Verifying a Run	7
10 Lab Server Setup (Alternative to Colab)	7
10.1 Step 1: Log In	7
10.2 Step 2: Clone the Repository	8
10.3 Step 3: Run the Setup Script	8
10.4 Step 4: Activate the Environment	8
10.5 Step 5: Run Experiments	8
10.6 Step 6: Keep Experiments Running After Logout	9
10.7 Step 7: Commit and Share Results	9
10.8 Quick Verification Checklist	9
10.9 Gotchas	9
11 Important Warnings	10
11.1 Anti-Disconnect Hack	10
11.2 Other Gotchas	10
12 Output Metrics	10
13 Quick Reference Card	11

1 Overview

LibSignal is an open-source reinforcement learning framework for **Traffic Signal Control (TSC)**. It wraps traffic simulators (SUMO, CityFlow) behind a unified interface so that different RL agents can be trained and evaluated on various road networks without rewriting glue code.

This repository continues the original DaRL-LibSignal/LibSignal project with Python 3.11+ compatibility, setup improvements, and realistic SUMO physics. The upstream project is no longer actively maintained; use this repository for all team work.

1.1 Key Concepts

Agent	The RL policy (e.g. DQN, PressLight, CoLight, MPLight).
World	The simulator backend (we use SUMO).
Network	The road topology and traffic demand (e.g. <code>sumo1x1</code> , <code>grid4x4</code>).
Episode	One full simulation run (default 3600 s = 1 hour of simulated traffic).
Action Interval	How often the agent makes a decision (default every 10 s).

2 Google Colab Setup

All experiments run on Google Colab. Each team member has a **personal output folder** on Google Drive so that checkpoints and logs are never lost when Colab disconnects.

2.1 Step 1: Open the Notebook

Open the shared Colab notebook. The first cell is the **Setup Cell**.

2.2 Step 2: Change Your Name

At the top of the setup cell, change `MEMBER_NAME` to your name:

```
MEMBER_NAME = "salman"    # Options: "salman", "rashid", "madina"
```

This determines your personal save folder on Google Drive:

Collective Intelligence Project/colab_<name>/output_data/

2.3 Step 3: Run the Setup Cell

Click the play button. The cell will:

1. Mount Google Drive.
2. Create your personal output folder (if it doesn't exist).
3. Install SUMO (first run only, takes ~2 minutes).
4. Clone or update LibSignal.
5. Install Python dependencies.
6. Create a **symlink** from the local `data/output_data` directory to your Drive folder, so all logs and model checkpoints are automatically saved to the cloud.

Warning

You **must** run the setup cell every time you open a new Colab session. Colab resets the virtual machine when the session ends, so the symlink and SUMO installation are lost.

2.4 Where Do Outputs Go?

When you run an experiment, outputs are saved to:

```
data/output_data/tsc/{world}_{agent}/{network}/{prefix}/
```

Because of the symlink, this maps directly to your Google Drive:

```
Drive/.../colab_<name>/output_data/tsc/sumo_presslight/grid4x4/test/
```

Inside that folder you will find:

- `logger/` — training logs (one `.log` file per run).
- `model/` — saved model checkpoints (`.pt` files, saved every 5 episodes by default).

3 Running Experiments

There are two ways to run experiments.

3.1 Option 1: Edit the Config File Directly (Recommended)

This is the most reliable method.

1. Open the Colab **file explorer** (folder icon on the left sidebar).
2. Navigate to `LibSignal/configs/tsc/`.
3. Open the YAML file for your agent (e.g. `presslight.yml`).
4. Edit the hyperparameters to match the values from the paper.
5. Save the file (`Ctrl+S`).
6. Run the experiment cell:

```
!cd /content/LibSignal && python run.py \  
  --task tsc \  
  --agent presslight \  
  --world sumo \  
  --network grid4x4 \  
  --prefix my_experiment_name \  
  --dataset onfly
```

3.2 Option 2: Smart Config Injector Cell

The notebook includes a “Smart Config Injector” cell that programmatically modifies the YAML config before each run and restores it afterwards. Edit the `OVERRIDES` dictionary in that cell.

```
OVERRIDES = {
    "model": {
        "learning_rate": 0.001,
        "batch_size": 64,
        "epsilon": 0.1,
        "epsilon_decay": 0.995,
    },
    "trainer": {
        "episodes": 200,
        "test_steps": 3600,
        "buffer_size": 10000,
    }
}
```

Tip

Use the `PREFIX` variable to give each run a unique name. This prevents experiments from overwriting each other. Example: `PREFIX = "lr001_eps01_bs64"`.

4 Command-Line Arguments Reference

All arguments are passed to `run.py`:

Flag	Default	Description
<code>--task / -t</code>	<code>tsc</code>	Task type. Always <code>tsc</code> for us.
<code>--agent / -a</code>	<code>dqn</code>	Agent name (must match a YAML file in <code>configs/tsc/</code>).
<code>--world / -w</code>	<code>cityflow</code>	Simulator. Use <code>sumo</code> .
<code>--network / -n</code>	<code>cityflow1x1</code>	Network config name (see Section 6).
<code>--dataset / -d</code>	<code>onfly</code>	Dataset type. Always <code>onfly</code> .
<code>--prefix</code>	<code>test</code>	Output subfolder name. Use this to tag experiments.
<code>--seed</code>	<code>None</code>	Random seed for reproducibility.
<code>--ngpu</code>	<code>-1</code>	GPU id (<code>-1</code> = CPU, <code>0</code> = first GPU).
<code>--interface</code>	<code>libsumo</code>	<code>libsumo</code> (fast) or <code>traci</code> (slow, supports GUI).
<code>--delay_type</code>	<code>apx</code>	<code>apx</code> (approximate) or <code>real</code> delay calculation.

Warning

Hyperparameters like `learning_rate`, `epsilon`, `batch_size`, `episodes`, etc. **cannot** be set from the command line. They must be set in the YAML config files or injected programmatically.

5 Available Agents

Agent Name	Config File	Type
maxpressure	maxpressure.yml	Heuristic baseline (no learning)
fixedtime	fixedtime.yml	Fixed-time baseline (no learning)
sotl	sotl.yml	Self-Organizing Traffic Light
dqn	dqn.yml	Deep Q-Network
presslight	presslight.yml	PressLight (pressure-based DQN)
mplight	mplight.yml	MPLight (FRAP-based, pressure)
colight	colight.yml	CoLight (graph attention network)
ppo	ppo.yml	Proximal Policy Optimization
frap	frap.yml	FRAP
maddpg	maddpg.yml	Multi-Agent DDPG

6 Available SUMO Networks

These are the `--network` values you can use with `--world sumo`:

Network Name	Description
sumo1x1	Synthetic single intersection
sumo1x3	Synthetic 1×3 arterial
sumo4x4	Synthetic 4×4 grid
sumo1x21	Synthetic 1×21 arterial
sumo7x28	Large 7×28 grid
sumohz1x1	Real-world Hangzhou single intersection
sumohz4x4	Real-world Hangzhou 4×4 grid
grid4x4	Alias for synthetic 4×4 grid (CityFlow-style name)
cologne1	Real-world Cologne single intersection
cologne3	Real-world Cologne 1×3 corridor

Info

For multi-intersection agents like **CoLight** and **MPLight**, use multi-intersection networks (`grid4x4`, `sumo4x4`, `cologne3`, etc.). Single-intersection networks work best for `dqn`, `presslight`, `ppo`, etc.

7 Configuration (YAML) Reference

Each agent has a YAML file in `configs/tsc/` that includes the base config. Agent-specific parameters override the base defaults. The inheritance chain is:

`configs/tsc/base.yml` → `configs/tsc/<agent>.yml`

7.1 Key Trainer Parameters

Parameter	Default	Description
episodes	200	Number of training episodes.
steps	3600	Simulation steps per training episode (seconds).
test_steps	3600	Simulation steps per test episode.
action_interval	10	Agent makes a decision every N seconds.
yellow_length	5	Yellow phase duration (seconds).
learning_start	5000	Steps before learning begins (replay buffer warm-up).
buffer_size	5000	Replay buffer capacity.
update_model_rate	1	Update model every N episodes.
update_target_rate	10	Update target network every N episodes.
test_when_train	True	Run a test episode after each training episode.

7.2 Key Model (Agent) Parameters

Parameter	Default	Description
learning_rate	0.001	Optimizer learning rate.
batch_size	64	Mini-batch size for training.
gamma	0.95	Discount factor γ .
epsilon	0.5	Initial exploration rate ϵ .
epsilon_decay	0.99	Multiplicative decay per episode.
epsilon_min	0.05	Minimum ϵ .
grad_clip	5.0	Gradient clipping threshold.
train_model	False	Set to True to enable training.
load_model	False	Load a previously saved checkpoint.

7.3 Logger Parameters

Parameter	Default	Description
save_rate	5	Save model checkpoint every N episodes.
save_model	True	Whether to save checkpoints at all.

8 Team Member Assignments

Check the Google Doc.

Each member should use the **PREFIX** flag to separate different runs:

```
--prefix run1_lr001_eps01
--prefix run2_lr0001_eps08
--prefix final_200ep
```

9 Verifying a Run

1. Run the setup cell.
2. Start a training run.
3. Wait ~2 minutes for the first episode to finish.
4. Open Google Drive in a new tab.
5. Navigate to: Collective Intelligence Project → colab_<name> → output_data.
6. You should see a folder tree like:

```
output_data/
  tsc/
    sumo_presslight/
      grid4x4/
        test/                # <-- your PREFIX
          logger/
            2026_02_10-16_39_57_BRF.log
          model/
            5_0.pt           # checkpoint at episode 5
            10_0.pt          # checkpoint at episode 10
            ...
```

7. If you see the model/ folder appearing, your setup is working correctly.

10 Lab Server Setup (Alternative to Colab)

Instead of Google Colab you can run experiments directly on the lab server. Each team member works from **their own clone** of the repository under their own home directory. There is no shared directory.

10.1 Step 1: Log In

Open a terminal and SSH in with the username and password Salman sent you on WhatsApp:

```
ssh rashid@<server-address>
# enter the password when prompted
```

Warning

Change your password immediately after first login:

```
passwd
```

Enter the old password, then choose a new one.

10.2 Step 2: Clone the Repository

Clone your own copy of the repo into your home directory:

```
git clone https://github.com/sal0-h/LibSignal.git ~/LibSignal
cd ~/LibSignal
```

10.3 Step 3: Run the Setup Script

Run the setup script with the `-no-sudo` flag. This installs Miniconda (if needed), creates the `traffic` conda environment, installs PyTorch, SUMO, and all Python dependencies.

```
chmod +x setup.sh
./setup.sh --no-sudo
```

The `-no-sudo` flag causes the script to skip any privileged system-package steps. The administrator has already installed the required system libraries, so the script will complete successfully without root access.

Info

You only need to run the setup script **once** after cloning. The environment persists across logins — unlike Colab, nothing is reset when you log out.

10.4 Step 4: Activate the Environment

Every new terminal session, activate the conda environment before running anything:

```
conda activate traffic
```

Verify it is working:

```
which python      # should point to ../envs/traffic/bin/python
python -V         # Python 3.10.x
```

10.5 Step 5: Run Experiments

Experiments are run exactly the same way as on Colab, just without the `!` prefix:

```
cd ~/LibSignal
python run.py \
  --task tsc \
  --agent presslight \
  --world sumo \
  --network sumo4x4 \
  --prefix rashid_run1 \
  --dataset onfly
```

Outputs are saved under your clone at:

`~/LibSignal/data/output_data/tsc/sumo_{agent}/{network}/{prefix}/`

There is no symlink to Google Drive; results stay on the server. Use `git add / commit / push` to share results with the team.

10.6 Step 6: Keep Experiments Running After Logout

Unlike Colab, the server does not kill your process when you disconnect. Use `nohup` or `tmux` so training continues in the background:

```
# option A: nohup (simplest)
nohup python run.py --task tsc --agent colight \
    --world sumo --network sumo4x4 \
    --prefix rashid_colight1 --dataset onfly \
    > run.log 2>&1 &

# then check progress at any time with:
tail -f run.log

# option B: tmux (recommended: you can re-attach the session)
tmux new -s train
python run.py ...           # start the run inside tmux
# detach: Ctrl+B then D
# re-attach later:
tmux attach -t train
```

10.7 Step 7: Commit and Share Results

After a run finishes, commit the output data (model weights are excluded by `.gitignore`; only logs and CSVs are committed):

```
cd ~/LibSignal
git add data/output_data/
git commit -m "rashid: presslight sumo4x4 run1 results"
git push
```

10.8 Quick Verification Checklist

1. You can SSH in and `passwd` has been changed.
2. `./setup.sh -no-sudo` completed with no errors.
3. `conda activate traffic` works.
4. A short test run completes:

```
python run.py --task tsc --agent maxpressure \
    --world sumo --network sumo1x1 --dataset onfly --prefix test
```

5. You see output in `data/output_data/tsc/sumo_maxpressure/sumo1x1/test/`.

10.9 Gotchas

- **Do not** run experiments with `sudo` — the conda environment does not live under root.
- **Do not** delete `~/miniconda3` or `~/.conda` unless you intend to redo the full setup.
- If you see `SUMO_HOME` not set, the conda env was activated *after* the setup script wrote the activation hook. Run `conda deactivate && conda activate traffic` to reload it.
- Model weight files (`*.pt`) are in `.gitignore` and will not be pushed. Only logs and metric CSVs are committed.
- Always use a unique `-prefix` per run to avoid overwriting previous results.

11 Important Warnings

Warning

Colab will disconnect if you close the tab. There is no way around this. Keep the tab open and prevent your computer from sleeping.

11.1 Anti-Disconnect Hack

To prevent Colab from disconnecting due to idle timeout (even while training is running), open the browser console (**Ctrl+Shift+I** → Console tab) and paste:

```
function ClickConnect(){
  console.log("Keeping Colab alive...");
  document.querySelector("colab-toolbar-button#connect").click()
}
setInterval(ClickConnect, 60000)
```

This clicks the “Connect” button every 60 seconds to prevent idle disconnection.

11.2 Other Gotchas

- **Do not** run `git pull` in the Colab repo directory if you have manually edited config files—it will overwrite your changes. Use the Smart Config Injector instead, which restores the original config after each run.
- **Do not** change `world.dir` in `base.yml`. It must remain `data/` for the output symlink to work.
- Colab free tier has a **~12 hour** session limit. For 200 episodes on a 4×4 grid, this should be enough, but monitor your progress.
- If you see `SUMO_HOME` errors, the setup cell didn’t run. Re-run it.

12 Output Metrics

The training log prints the following metrics each episode:

Metric	Meaning
<code>q_loss</code>	The Q-network loss (should decrease over training).
<code>rewards</code>	Average reward per step (higher is better).
<code>queue</code>	Average queue length at intersections (lower is better).
<code>delay</code>	Average vehicle delay (lower is better).
<code>throughput</code>	Number of vehicles that completed their trip (higher is better).
<code>real avg travel time</code>	Mean travel time of all vehicles (lower is better). This is the primary metric for comparison.

13 Quick Reference Card

Cheat Sheet

```
# Baseline (no learning, instant)
python run.py -a maxpressure -w sumo -n sumo1x1 --prefix baseline

# DQN on single intersection
python run.py -a dqn -w sumo -n sumo1x1 --prefix dqn_run1

# PressLight on 4x4 grid
python run.py -a presslight -w sumo -n grid4x4 --prefix pl_run1

# CoLight on 4x4 grid
python run.py -a colight -w sumo -n grid4x4 --prefix col_run1

# MPLight on Cologne corridor
python run.py -a mplight -w sumo -n cologne3 --prefix mpl_run1
```

All commands assume: `--task tsc --dataset onfly`